

Week 2 Day 4 — CrewAI: Multi-Agent Collaboration, Roles & Task Delegation

Notebook write-up and review | Due 10 Sept 2026

This document summarizes the submitted notebook (*week2_day4.ipynb*), reviews it against the assignment's five tasks, and gives setup instructions to reproduce the run. The notebook builds a 3-agent CrewAI crew that reviews a sales dataset, generates business insights, and writes a stakeholder-ready summary — first with CrewAI's sequential process, then with a hierarchical (manager-delegated) process — and logs token usage and cost for both.

Business Task Chosen

Review a sales dataset, generate business insights, and write a stakeholder-ready summary. This mirrors how a data team hands off work across specialists: one person cleans/profiles the data, one turns numbers into insights, and one writes it up for a non-technical audience. A small 9-row sample sales CSV (month, region, product, units_sold, revenue) is generated in-notebook so the workflow runs end-to-end without any external file.

Task 1 — Multi-Agent Design Thinking

Three roles, no overlap:

- **Data Analyst** — reads the raw CSV, produces a verified statistical profile (totals/trends by month, region, product; flags anomalies).
- **Business Insight Generator** — interprets the profile into 3–5 business-relevant insights with likely causes.
- **Stakeholder Report Writer** — turns the insights into a concise, non-technical executive summary with recommended actions.

Why multiple specialists can beat one generalist: a single agent holding "clean data" + "form a narrative" + "write for execs" in one prompt pulls the model toward different tones and priorities in the same context, raising the odds a step gets diluted or skipped. Splitting into roles gives each step a tight, role-scoped prompt and lets you re-run just the failing step instead of the whole chain.

Where it isn't worth it: trivial datasets (a handful of rows/columns) or one-off exploratory questions — the coordination and token overhead of agents talking to each other costs more than it saves versus one well-prompted agent doing everything in a single pass.

Task 2 — Agents & Role-Appropriate Tools

Agent	Tools	Justification
Data Analyst	FileReadTool	Only agent that touches raw data — reads the CSV directly. Reading the raw file needs a tool
Insight Generator	None	Works only from the analyst's verified JSON profile — kept from re-deriving its own (possibly c

Report Writer	None	Pure writing task performed on the insight list; no data access needed.
---------------	------	---

All three agents share one Gemini LLM config (gemini-3.5-flash-lite, temperature 0.3). Embeddings are also pointed at Gemini so no OpenAI key is required anywhere in the notebook.

Task 3 — Tasks, Sequential Process & Log Review

Each CrewAI Task has a clear description + expected_output, and a context=[...] link to the prior task so outputs chain through the crew: Profile → Insights → Report. Assembled with Process.sequential and run via crew.kickoff_async() (Jupyter needs nest_asyncio.apply() first, since CrewAI's async LLM calls otherwise conflict with Jupyter's own event loop).

Format-mismatch found & fixed:

In an early run, the analyst's raw output included a leading sentence ("Here is the profile:") before the JSON, which broke the Insight Generator's downstream parsing. **Fix:** tightened expected_output to say *"Return ONLY the JSON, no prose around it"* — pushing formatting constraints into expected_output rather than description, since that's what the agent self-checks its final answer against.

The sequential run completed cleanly end-to-end: the Data Analyst produced a JSON profile (totals by month/region/product plus the single biggest month-over-month change), the Insight Generator turned that into a numbered list of business insights, and the Report Writer produced a markdown executive summary under 300 words with a headline and recommended actions.

Task 4 — Hierarchical Delegation

The same three specialist agents are reused; a fourth **Analytics Team Manager** agent (allow_delegation=True) is added and the crew is rebuilt with Process.hierarchical and manager_agent=... The manager delegates each step to the right specialist, reviews the hand-off against the brief, and can send work back for revision before passing it on.

Sequential vs Hierarchical — comparison

Aspect	Sequential	Hierarchical
Quality	Consistent, predictable hand-offs	Can catch/fix a bad hand-off via manager review
Latency / cost	Lower — no manager overhead	Higher — manager adds extra LLM calls per step
Reliability	Fails silently if one agent's output is malformed	More robust — manager can request a redo
Best for	Well-defined, low-ambiguity pipelines	Tasks needing quality control / dynamic delegation

Task 5 — Evaluation & Cost Awareness

Both runs were logged via crew.usage_metrics with an approximate cost calculation (price per 1K input/output tokens). Actual figures from the notebook's own executed run:

Run	Wall-clock time	Total tokens	Requests	Approx. cost
Sequential	50.6 s	9,825	12	\$0.00782
Hierarchical	373.9 s (~6.2 min)	170,096	112	\$0.11834

The hierarchical run used roughly **17x** the tokens and **15x** the cost of the sequential run for the same underlying task, driven by the manager's extra delegation/review round-trips (112 vs 12 successful requests).

Success criteria (defined in notebook)

- Factual grounding — every number in the final report traces back to the analyst's JSON profile.
- Completeness — report covers month, region, and product trends, not just one dimension.
- Tone — report reads as non-technical/executive-friendly, under 300 words.

The notebook prints a manual scoring sheet (1–5 per criterion) for each run, to be filled in after reading the printed output. **These scores are currently left blank in the submitted notebook** — see review notes below.

Review Notes

The notebook was checked cell-by-cell: all 19 cells execute in order with no kernel errors or tracebacks anywhere in the outputs. Both the sequential and hierarchical crews ran to completion and produced final answers. Structure and content map cleanly onto all five tasks in the brief, including the Task 2 tool justifications, the Task 3 documented format-fix, the Task 4 manager agent and comparison table, and the Task 5 token/cost logging.

Outstanding before submission:

- The manual scoring sheet (Task 5) has blank "___/5" scores for both runs — read the printed sequential and hierarchical outputs in the notebook and fill these in.
- The brief also asks for 3–4 sentences on whether the multi-agent crew was worth the added complexity/cost for this task — this written verdict is not yet in the notebook and should be added, informed by the cost table above (roughly 15x cost for the hierarchical version).
- Optional: a comparison against the Day 3 single-agent LangGraph solution is mentioned as "if applicable" in the brief — add it if that solution's token/cost numbers are available.

Setup — Reproducing the Run

1. Create a virtual environment (recommended)

```
python3 -m venv venv && source venv/bin/activate
```

2. Install dependencies

```
pip install crewai crewai-tools nest_asyncio --break-system-packages
```

3. Get a Gemini API key

Create a free key at Google AI Studio (aistudio.google.com/apikey). This is the only key needed — the notebook avoids CrewAI's OpenAI-embedding RAG tools, so no OPENAI_API_KEY is required.

4. Set the key as an environment variable

```
export GEMINI_API_KEY="your-key-here"
```

5. Launch Jupyter and run all cells top to bottom

jupyter notebook week2_day4.ipynb → Cell → Run All. The nest_asyncio patch in the first code cell is required — without it, crew.kickoff_async() raises an event-loop error inside Jupyter's kernel.

6. Expect a few minutes and a small cost

Based on the notebook's logged run: sequential ≈ 51 s / \$0.008; hierarchical ≈ 6 minutes / \$0.12, since the manager agent adds many extra delegation and review calls.